шаурматч_

Платформа обратного ИТ-найма на подтверждённых навыках

Сопроводительная документация · хакатон «Лидеры цифровой трансформации 2026», спецтрек ФСП

Сборка 10.10.2026 · источник — docs/*.md и docs/openapi.json в репозитории

Содержание

1. Обзор решения и быстрый старт
2. Функциональная и компонентная архитектура
3. Механика тестирования и подбора: метод «ШАУРМА»
4. Валидация метода: процедура и результаты
5. «Маринад»: обучение на данных платформы
6. «Маринад»: отчёт симуляции
7. Интеграция с реестром ФСП
8. Описание API
9. Библиотеки и компоненты
10. Сборка, развёртывание и локальный запуск

1. Обзор решения и быстрый старт

Хакатон «Лидеры цифровой трансформации 2026», спецтрек Федерации спортивного программирования (ФСП).

Кандидат не откликается вслепую. Его категория (**специализация × грейд**) определяется **опросом, выбором грейда и тестом**, который невозможно «слить», — метод **«ШАУРМА»**. Работодатель описывает потребность, получает подборку из подходящих категорий с объяснением и **сам выходит на контакт** с приглашением, где **зарплата «от–до» указана обязательно**. Контакты кандидата открываются только после того, как он принял приглашение.

Метод тестирования и подбора	docs/МЕТОД ШАУРМА.md
Валидация метода (цифры, графики)	docs/validation/report.md
«Маринад»: как платформа учится на своих данных	docs/МАРИНАД.md · отчёт docs/training/report.md · страница `/marinade`
Архитектура	docs/АРХИТЕКТУРА.md
Интеграция с реестром ФСП	docs/ИНТЕГРАЦИЯ ФСП.md
API (OpenAPI 3)	docs/openapi.json · Swagger UI: `/api/docs`
Сборка и развёртывание	docs/РАЗВЁРТЫВАНИЕ.md
Библиотеки и версии	docs/БИБЛИОТЕКИ.md

Быстрый старт

```
cp .env.example .env          # при необходимости поменяйте порт и PUBLIC_URL
docker compose up -d --build
```

Откройте **http://localhost** (порт задаётся `HTTP_PORT`). При первом старте база заполняется демо-данными: 360 синтетических кандидатов прошли ту же механику ШАУРМА, есть 5 компаний, потребности, вакансии и приглашения.

Демо-аккаунт (пароль `shaurma2026`)	**Что показать**
`new@shaurmatch.demo`	Путь кандидата с нуля: опрос → тест → категория
`alexey@shaurmatch.demo`	Кандидат BE·SEN с историей ФСП, входящие приглашения
`maria@shaurmatch.demo`	Кандидат FE·MID **без истории ФСП**
`employer@shaurmatch.demo`	Работодатель «ТехноЛаб»: подбор, карта рынка, воронка, «Слепая дегустация»

Можно и зарегистрироваться самому: в демо-режиме (`DEMO_MODE=true`) ссылка подтверждения email показывается на экране. Все письма также видны в Mailpit: `http://localhost:8025`.

Сквозной сценарий (ТЗ, п. 3.3)

1. **Кандидат:** регистрация по email → подтверждение → опрос (отрасль, специализация, стек) → выбор грейда → адаптивный тест → категория (например `BE·MID`). Если грейд не подтвердился, можно сразу пройти тест на уровень ниже. Смена грейда — не чаще раза в 90 дней. Можно добавить дополнительные категории (по тесту в другой специализации), например `BE·SEN` + `DS·MID` — у демо-кандидата `alexey@shaurmatch.demo`.
2. **Работодатель:** «Подбор» → описать потребность словами → рецепт навыков → рекомендованные категории и ранжированная подборка с объяснением → «Пригласить» (вилка обязательна).
3. **Кандидат:** «Приглашения» → принять или отклонить с причиной. После принятия контакты открыты работодателю.
4. **Регулярные задачи:** раз в неделю система адресно предлагает кандидату задачу — сначала от работодателя его категории («Слепая дегустация»), а если таких нет, «Свежую начинку» по «остывшему» или слабому навыку. Решить или предложить подход; проверка автоматическая. **Профиль:** ФИО и места работы видны работодателю по умолчанию (контакты — только после согласия), стек подтверждается заданиями теста, разбор PDF-резюме извлекает роли и места работы, а своя модель подсказывает специализацию.
5. **«Маринад»:** ответы кандидатов калибруют тест, правки рецептов учат языковую модель, ответы на приглашения и отметки «наняли» учат подбор (P(подойдёт) × P(примет)). Всё видно на `/marinade`.
6. **Дополнительно:** вакансии и отклики, регулярные задачи («Слепая дегустация»), «Честный ценник» против фиктивных вакансий, PDF-профиль, режимы доступности, мобильная версия.

Структура репозитория

```
engine/      ШАУРМА-движок (Python, FastAPI): шаблоны, мутации, адаптивный тест, Glicko-2, рецепты, матчинг, антислив, валидация,
             «Маринад» (app/learn: калибровка, детектор дрейфа, ранжировщик, языковая модель; training/: симуляция года)
backend/     REST API (Node.js, Express, TypeScript, PostgreSQL), OpenAPI, авторизация, Keycloak (OIDC)
frontend/    Веб-приложение (React, TypeScript, Vite), дизайн-система в src/styles.css
fsp-mock/    Мок-сервис реестра ФСП (структура данных предложена командой)
keycloak/    Realm «fsp» для входа через ФСП ID (необязательный профиль)
docs/        Документация, отчёт валидации, openapi.json, брендбук и логотипы (docs/brand)
scripts/     Сборка PDF-документации
```

Локальная разработка без Docker

```
docker run -d --name proof-pg -e POSTGRES_USER=proof -e POSTGRES_PASSWORD=proof -e
POSTGRES_DB=proof -p 5433:5432 postgres:16-alpine
cd engine   && pip install -r requirements.txt && uvicorn app.main:app --port 8001
cd fsp-mock && npm i && node server.js
cd backend  && npm i && npm run seed && npm run dev      #
http://localhost:8080/api/docs
cd frontend && npm i && npm run dev                      # http://localhost:5173
cd engine   && python -m validation.simulate --n 2000    # пересчитать отчёт валидации
cd engine   && python -m training.marinate --bake         # «год жизни платформы» +
запечь модели «Маринада»
```

Код открыт и не обфусцирован. Компании и демо-профили вымышлены.

2. Функциональная и компонентная архитектура

1. Функциональная архитектура

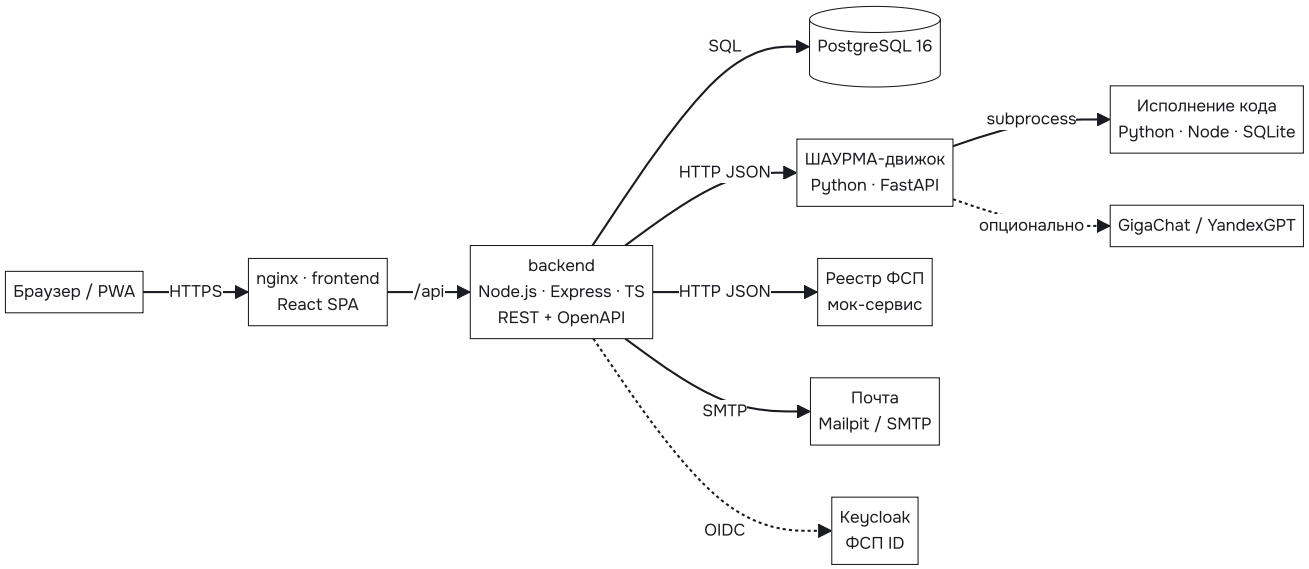
Роль	Функции	Где в коде
Кандидат	Регистрация по email с подтверждением; профиль вручную или из PDF-резюме (автораспознавание); стандартизированный PDF-профиль; опрос по отрасли и специализации; выбор грейда; адаптивный тест; категория и история; тест на грейд ниже или выше (не чаще раза в 90 дней); входящие приглашения (принять, отклонить с причиной); «Ценник не врёт»; вакансии и отклики; регулярные задачи; приватность и согласия; связь с ID ФСП	`backend/src/routes/candidate.ts`, `tests.ts`, `invitations.ts`, `vacancies.ts`, `tasks.ts`
Работодатель	Профиль компании; описание потребности → рецепт; подборка по категориям с объяснением; уточняющие фильтры поверх подборки; банк кандидатов с фильтрами (специализация, грейд, категория, стек, ФСП); карта рынка; приглашение с вилкой без обязательной вакансии; статусы приглашений; вакансии и отклики; «Слепая дегустация»	`backend/src/routes/employer.ts`, `vacancies.ts`, `tasks.ts`
ШАУРМА-движок	Генерация уникальных вариантов (эталон получается исполнением кода); сложность по LLTM; адаптивный тест и решение по грейду; Glicko-2 по ингредиентам; рецепт по тексту; матчинг с объяснением; канарейки и person-fit; разбор резюме; валидация	`engine/app/*`, `engine/validation/simulate.py`
Реестр ФСП (мок)	Участник по ID: дисциплины, соревнования, уровень, место, число участников, команда, роль, разряд	`fsp-mock/server.js`

Правила видимости (ТЗ, п. 2.2)

- Зарплата «от–до» в ₽ обязательна в вакансии и в приглашении: это проверки на уровне схемы API (zod) и ограничения `CHECK` в БД.
- Контакты кандидата (email, телефон, Telegram, ФИО) работодатель получает **только** если кандидат принял его приглашение или откликнулся на его вакансию (`services/candidate.ts → contactsVisible`). До этого показывается «Кандидат #код». Город и зарплату кандидат может скрыть в настройках.

- Кандидат видит только свои приглашения и отклики: все запросы фильтруются по `candidate_id = req.user.id`.
- Статусы приглашений и откликов: `sent → viewed → accepted | declined`.
- Выдача формируется только из кандидатов с категорией, присвоенной тестом (`grade IS NOT NULL`) и согласием на публикацию профиля.

2. Компонентная архитектура



- **Монолит с явным разделением слоёв плюс отдельный вычислительный сервис.** Backend отвечает за данные, права и бизнес-правила. Движок не хранит состояния: он только считает, поэтому масштабируется горизонтально.
- **Эталон без нейросети.** Правильный ответ каждого варианта получается исполнением сгенерированного кода (Python, Node.js, SQLite). LLM используется только для необязательного уточнения рецепта и разбора резюме, а без ключей система работает на правилах.
- **Хранение.** Реляционная PostgreSQL. Рейтинги Glicko-2 по ингредиентам хранятся в `jsonb` у кандидата, варианты заданий — в `test_items` (это нужно для проверки канареек и аудита).

Модель данных (основное)

`users` → `candidates` (1:1) | `companies` (1:1); `candidate_categories` (все подтверждённые тестом категории кандидата: основная + дополнительные; представление `candidate_category_view` — одна строка на кандидата и категорию, по нему ищет работодатель); `test_sessions` → `test_items`; `grade_history`; `needs` (рецепт `jsonb`); `vacancies` → `applications`; `invitations`; `tasks` → `task_submissions` (`tasks.kind`: `employer` — «Слепая дегустация» для категории, `system` — «Свежая начинка», адресная задача кандидату). Схема: `backend/migrations/001_init.sql`.

«Маринад»: обучение на данных платформы

Backend (`services/learning.ts`) решает, когда пора учиться: каждые 300 ответов на задания, 20 ответов на приглашения, после каждой правки рецепта. Он собирает данные из БД и вызывает движок (`/learn/*`). Движок обучает модель и пишет её как открытый JSON в docker-том `models`. Все воркеры подхватывают новую версию по времени изменения файла. Журнал версий хранится в таблице `model_versions`, состояние триггеров — в `learning_state`. Обучение идёт в фоне: пользователь не ждёт. Подробно: МАРИНАД.md.

3. Безопасность и 152-ФЗ

- Пароли хешируются bcrypt. JWT лежит в httpOnly-cookie (SameSite=Lax), также поддерживается `Authorization: Bearer`.
- Роли проверяются на каждом методе (`requireRole`). Все SQL-запросы параметризованы.
- Ограничение частоты запросов на вход, регистрацию и подтверждение email. Используется helmet, CORS разрешён только для `PUBLIC_URL`.
- Согласие на обработку ПДн обязательно перед тестом. Согласие на публикацию обезличенного профиля — отдельно. Дата согласия сохраняется.
- Минимизация данных: работодатель видит категорию, рейтинги и стек без контактов, пока кандидат не дал согласие на контакт.
- Модели. Тест, подбор и разбор текста работают на собственных открытых моделях шаурматч_ (без внешних API). LLM — необязательная надстройка: адаптер поддерживает GigaChat и YandexGPT, организаторы допускают и зарубежные модели (предпочтительнее открытые). В LLM не передаются контакты; разбор резюме по LLM включается только при наличии ключа.

4. Производительность

- Подборка по потребности строится за один запрос к БД по индексу `(specialization, grade)` и один вызов движка (векторная арифметика по сотням кандидатов). При 360+ кандидатах это десятки миллисекунд.
- Генерация варианта: 1–5 запусков кода, около 50 мс каждый, с LRU-кэшем. Шаг адаптивного теста — EAP на сетке numpy, около 1 мс.
- Движок работает с двумя воркерами uvicorn. Backend и движок масштабируются репликами, так как состояние хранится в PostgreSQL.

5. Интеграции

- **ФСП:** см. ИНТЕГРАЦИЯ ФСП.md.
- **ATS (концепция):** приглашения и статусы публикуются вебхуком (`invitation.created`, `invitation.status_changed`) в ATS компании (Хантфлоу, Potok, E-Staff). Импорт вакансий из ATS идёт по их API. Адаптер — отдельный модуль backend по шаблону `services/fsp.ts`.
- **Защита от фиктивных вакансий («Честный ценник»):** лимит приглашений в неделю (5 + репутация/10), талон возвращается после ответа кандидата. Кандидат подтверждает, совпала ли зарплата с вилкой: «нет» снижает репутацию на 25 и снимает бейдж. Дальнейшее развитие — сигналы фиктивности: вакансия без найма месяцами, одинаковые тексты, вилка-приманка.

3. Механика тестирования и подбора: метод «ШАУРМА»

> **«Задачу можно слить. Шаурму — нет: каждую заворачивают заново, при тебе и по твоему заказу».**
>
> ШАУРМА — подход платформы «шаурматч_» к тестированию, категоризации и подбору ИТ-специалистов. Он закрывает два главных критерия ТЗ ФСП: механику подбора (35%) и механику тестирования, валидации и категоризации (25%).

---

0. Суть в одном абзаце

У всех существующих сервисов **тест, профиль кандидата и вакансия описаны на трёх разных языках**. Тест выдаёт балл, резюме — список слов, вакансия — текст. Поэтому подбор сводится к сравнению ключевых слов, а тест «прикручен сбоку».

ШАУРМА вводит **единое «Меню ингредиентов»**: каталог элементарных навыков ИТ-специалиста, например «граничные случаи», «асинхронность», «индексы БД», «сложность алгоритма». Один и тот же ингредиент одновременно: 1. **задаёт сложность задачи** — каждый ингредиент добавляет известную долю сложности (линейно-логистическая модель теста, LLTM); 2. **измеряется у кандидата** — у каждого ингредиента своя «температура освоения» (когнитивная диагностика по Q-матрице «задача → навыки» плюс рейтинг Glicko-2); 3. **заказывается работодателем** — вакансия превращается в «заказ»: какие ингредиенты нужны и в какой пропорции.

Задача, человек и вакансия оказываются **в одном пространстве**. Тест собирается по заказу, кандидаты ранжируются по тем же ингредиентам, которые проверял тест, а объяснение выдачи получается само: «из 5 ингредиентов заказа у кандидата подтверждены 4».

---

1. Аналоги: что существует и почему этого мало

1.1 Обратный найм (работодатель пишет первым)

Сервис	Что было	Чем закончилось и почему
Hired (США)	Компании отправляли кандидатам приглашения на интервью с зарплатой; оценка ~$500 млн	Почти закрылся в 2020-м, куплен Vettery (Adecco); в 2024-м свёрнут в **LHH** — тот самый ориентир из ТЗ. Нет проверки навыков, бизнес держался только на «горячем» рынке
Vettery	То же, кураторский отбор	Слился с Hired и ушёл в LHH

Сервис	Что было	Чем закончилось и почему
Triplebyte	Собственный технический скрининг и витрина проверенных инженеров	В 2023-м витрину закрыли, технологию тестов купила Karat. Погоня за объёмом размыла качество, плюс скандалы с приватностью
RailsDevs и другие «обратные доски»	Нишевые профили, компании платят за контакт	Профили «съедаются» наймом, нужно постоянно набирать новых, а доверие не масштабируется
Otta, Cord, Underdog	Профиль с ожиданиями, компании выходят на кандидата	Нет объективной проверки навыков
getmatch, Хабр Карьера, hh.ru (РФ)	Вакансии с открытыми зарплатами, база резюме, рекрутеры	Классический отклик. Работодатель всё равно вручную фильтрует отклики

Вывод: обратный найм без **проверенных навыков** превращается в спам по базе, а площадка теряет кандидатов. Нужна проверка, которая сама поддерживает профиль «живым».

1.2 Проверка навыков

Сервис	Подход	Слабое место
LinkedIn Skill Assessments	Фиксированные тесты с бейджами	**Закрыты в 2023-м**: ответы ко всем тестам лежали на GitHub. Это ровно та угроза, о которой пишет ТЗ
CodeSignal, HackerRank, Codility	Закрытый банк, ротация, прокторинг, мониторинг утечек	Утечку только замедляют, банк приходится держать в секрете вечно
HackerRank, TestGorilla — «тест из описания вакансии»	ИИ подбирает вопросы из библиотеки по тексту вакансии	Вопросы фиксированные. Сложность сгенерированных ИИ вопросов не откалибрована. Результат — общий балл, а не профиль навыков
Karat (бывший Triplebyte)	Адаптивные скрининги и интервью	Инструмент для компаний, а не витрина кандидатов. Дорого
Хабр Карьера	Тесты по навыкам → отметка в профиле	Фиксированные тесты, балл не связан с подбором
Генерация задач нейросетью	Каждому уникальная задача	Сложность несопоставима, ответ может быть галлюцинацией (ТЗ называет это прямо)
Codeforces и спортивные рейтинги	Объективный рейтинг, ценится частью рекрутеров	Мерит только спортивные алгоритмы, не связан с найм-процессом
GapJumpers («слепые прослушивания»)	Анонимное задание = отклик	Инициатива у кандидата, компании не конкурируют за кандидата

1.3 Что берём из науки (проверенные блоки)

- **Автоматическая генерация заданий (AIG)** — из одной «родительской» модели получают много вариантов.
- **LLTM (линейно-логистическая модель теста, Фишер)** — сложность задания = сумма сложностей его операций, поэтому сложность нового задания известна заранее.
- **Когнитивная диагностика (DINA, Q-матрица)** — из ответов восстанавливают, какие именно навыки освоены.
- **Адаптивное тестирование (CAT) и контроль частоты показа заданий** (randomesque, Sympson–Hetter).
- **Glicko-2** — рейтинг с неопределённостью, которая растёт при простое. На Lichess так калибруют шахматные задачи «игрок против задачи».
- **Canary trap** — разные версии документа, чтобы найти источник утечки. Плюс поиск «заранее известных» заданий по статистике ответов.

Чего мы не нашли ни у кого: чтобы эти блоки работали **в одном пространстве ингредиентов**, общем для задачи, кандидата и вакансии, внутри площадки обратного найма. Именно это и есть ШАУРМА.

---

2. Почему именно шаурма

Шаурма	Метод
Лаваш одинаковый у всех и лежит на виду	Базовые шаблоны задач **открыты**. Секретов нет, сливать нечего
Начинку кладут каждый раз заново	Каждый кандидат получает **уникальный вариант**: мутации шаблона по своему seed
Заворачивают при тебе, кухня открыта	**Прозрачность:** кандидат видит рецепт (какие ингредиенты проверяются), работодатель видит, почему кандидат в подборке
Готовится по заказу: «без лука, поострее»	**Вакансия — это заказ.** Из неё собирается тест и регулярная задача
В Москве шаурма, в Питере шаверма — суть одна	**Изоморфные задачи:** разные названия и обёртки, одна и та же логика и сложность
Вертел крутится без остановки, мясо срезают свежим	**Рейтинг обновляется непрерывно.** Без активности «мясо остывает»: растёт неопределённость, и профиль опускается в выдаче
У каждого ларька свой фирменный соус	**Канарейки:** у каждого варианта свои метки, по ним видно, чья это «шаурма»
Съеденную шаурму не перепродашь	Вариант одноразовый: **ответ к чужому варианту бесполезен** и разоблачает списавшего
У ларька все равны: студент, сеньор, директор	**Слепой отбор** без вуза, возраста, пола, фото. Инклюзивность
Еда хакатонов, ночных дедлайнов и программистов	Понятна целевой аудитории, легко запоминается жюри
Цена висит на табличке до заказа	**Зарплата видна до начала общения** — принципиальное требование ТЗ

Шаурма	Метод
Бургер собирают заранее и держат под лампой	Обычный тест — заранее собранный набор, который «лежит» и протухает (сливается)

И это просто акроним из шести частей метода:

	Часть	Шаурма-термин
Ш	**Шаблоны** — открытый банк	Лаваш
А	**Адаптивная сборка по заказу** — вакансия → рецепт → адаптивный тест	Заказ
У	**Уникальная начинка** — мутации с известной сложностью	Начинка
Р	**Рейтинг на вертеле** — Glicko-2 по каждому ингредиенту	Вертел
М	**Матчинг вслепую** — «Слепая дегустация» и «Честный ценник»	Окошко ларька
А	**Антислив** — канарейки и статистика соответствия ответов	Фирменный соус

---

3. Меню ингредиентов (ядро метода)

Ингредиент — элементарный, проверяемый навык. Справочник составляем сами (ТЗ это разрешает). Ингредиенты сгруппированы по специализациям:

Группа	Примеры ингредиентов
Общие	Граничные случаи, сложность алгоритма, чтение чужого кода, отладка, тестирование
Backend	REST-контракты, транзакции и изоляция, индексы БД, очереди и идемпотентность, кэширование, конкурентность
Frontend	Состояние и рендер, асинхронность в UI, доступность (a11y), производительность рендера
Data / ML	Подготовка данных, утечка таргета, метрики качества, SQL-агрегации
DevOps	Контейнеры, CI/CD, наблюдаемость

У каждой задачи есть **Q-строка**: какие ингредиенты она требует. Каждый **оператор мутации** добавляет ингредиент: например, «пустой вход» добавляет «граничные случаи», а «параллельный вызов» — «конкурентность».

---

4. Ш — Шаблоны: открытый лаваш

- Базовые задачи публикуются **открыто** вместе с разбором. Готовиться по ним честно и полезно.
- Шаблон — это задача с параметрами и точками мутации плюс Q-строка ингредиентов.
- Форматы на MVP (все проверяются автоматически):
- **Чтение кода:** что выведет программа или какое значение вернёт функция.

- **Найди дефект:** какая строка неверна и почему.
- **Короткий код:** решение проверяется автотестами в песочнице.
- **Подход к решению** (открытый ответ, используется в регулярных задачах): ИИ оценивает по рубрике, в рейтинг идёт с малым весом.

5. A — Адаптивная сборка по заказу

5.1 Вакансия → рецепт

ТЗ прямо просит показать, как по описаниям вакансий система формирует задания: 1. Работодатель пишет потребность обычным текстом. 2. ИИ (GigaChat / YandexGPT) раскладывает текст на **специализацию, грейд и вектор ингредиентов с весами**: «Нужен бэкендер на Go, highload, Kafka» → `очереди и идемпотентность 0.3, конкурентность 0.3, индексы БД 0.2, …`. Работодатель видит рецепт и может поправить веса ползунками. 3. Рецепт задаёт, какие шаблоны и мутации войдут в **регулярную задачу компании** и в **дополнительный блок теста** для подходящих кандидатов.

5.2 Первичный тест (обязательный путь по ТЗ)

1. **Опрос:** отрасль → специализация → стек.
2. **Самооценка:** кандидат сам выбирает предполагаемый грейд.
3. **Адаптивный тест:** старт у порога заявленного грейда. Следующее задание выбирается по максимуму информации **и покрытию ингредиентов специализации**. Частота показа задач ограничена: случайный выбор из 5 лучших заданий и лимит для каждого шаблона. Тест останавливается, когда точность достаточна, или после 12–15 заданий.
4. **Решение по грейду:** - P(уровень ≥ порог заявленного грейда) ≥ 0.55 → грейд подтверждён (порог подобран валидацией: максимум к при доле завышенных грейдов < 6%); - P ≥ 0.8 для следующего грейда → кандидату сразу предлагается тест на повышение; - грейд не подтверждён → можно сразу пройти тест на уровень ниже. **Принудительного понижения нет.**
5. **Смена грейда — не чаще раза в 90 дней.**
6. **Категория = специализация × подтверждённый грейд** (`BE·MID`) — её видит работодатель. Стек и формат работы — фильтры, а не часть кода.
7. Побочный результат теста — **профиль ингредиентов**: какие навыки освоены и насколько уверенно.
8. **Банк заданий покрывает все грейды.** Шесть лёгких шаблонов (b ≈ -1.6…-0.6: срезы и словари в Python, filter/map и строки в JS, WHERE/ORDER BY/LIMIT в SQL, простой дефект) дают тесту точку опоры ниже порога Junior. Без них стажёру тест подбирал задания уровня Middle, и оценка была грубой.
9. **Несколько категорий.** После подтверждения основной категории можно добавить дополнительные, по одной на специализацию, — каждую своим адаптивным тестом (например, BE·SEN + DS·MID). У каждой категории свои правила: грейд не понижается принудительно, смена — не чаще раза в 90 дней. Рейтинги ингредиентов общие (SQL-навыки бэкендера засчитываются и в Data). Работодатель видит кандидата в подборке и банке каждой его категории, карточка открывается в разрезе специализации

потребности. Смена основной специализации в опросе не переносит грейд: в новой специализации он есть, только если подтверждён тестом.

6. У — Уникальная начинка

Вариант = **шаблон + seed + операторы мутации + обёртка**.

Слой	Пример	Кто делает
Параметры	Размеры входа, граничные значения	Детерминированный генератор по seed
Операторы мутации	Смещение на единицу (off-by-one), инвертированное условие, пустой вход, переполнение, параллельный вызов, другая структура данных	Детерминированный генератор
Обёртка («шаурма / шаверма»)	Тот же алгоритм в контексте логистики, банка, турнира ФСП	GigaChat / YandexGPT, только текст

- **Правильный ответ получается исполнением кода в песочнице, а не нейросетью.** Ошибка ИИ не может стать «правильным ответом».
- **Сложность варианта известна до показа (LLTM):** `b(вариант) = b(шаблон) + Σ η(оператор) + ε(обёртка ≈ 0)`. Сложности η операторов уточняются по накопленным ответам. Поэтому **разные кандидаты получают разные, но сопоставимые по сложности задания**. Это прямой ответ на проблему, которую ставит ТЗ.
- **Контроль обёртки:** после перефразирования вариант снова прогоняется через эталон и автотесты. Если смысл изменился, вариант отбрасывается.

7. Р — Рейтинг на вертеле

- Каждое решение — **матч «кандидат против задачи»**. Задачи тоже имеют рейтинг и калибруются онлайн, как шахматные задачи на Lichess, без отдельного предварительного прогона.
- **Glicko-2 считается по каждому ингредиенту**, а не одним числом. У каждого ингредиента три величины: сила μ, неопределённость RD и стабильность σ.
- **Мясо остывает:** без активности RD растёт, профиль теряет позиции. **Регулярные задачи** (из ТЗ) снижают RD, профиль снова «горячий». Так закрывается требование ТЗ об актуальности профиля.
- **Общая сила в категории** — консервативная оценка **μ − 2·RD** («кандидат не слабее этого»), взвешенная по ингредиентам специализации.
- **Достижения ФСП** засчитываются как внешние подтверждённые матчи по ингредиентам дисциплины. Например, продуктовое программирование даёт «REST-контракты», «командную разработку», «граничные случаи». Вес зависит от уровня соревнования, места и роли.
- **Без истории ФСП штрафа нет:** профиль подтверждается тестами. В интерфейсе нейтральная пометка «Подтверждено тестами платформы».

8. M — Матчинг вслепую

8.1 Подборка по заказу (основной путь по ТЗ)

```
Score(кандидат | заказ) = Σ_i  w_i · Mastery_i(μ_i - 2·RD_i)      ← совпадение ингредиентов
                        × Fit(стек, формат, ожидания по зарплате)  ← жёсткие условия и стек
                        × (1 + бонус ФСП)
```

- Выдача строится из **категорий, присвоенных тестом**, а не из резюме (требование ТЗ).
- **Объяснение у каждой позиции:** «из 5 ингредиентов заказа у кандидата подтверждены 4. Самый сильный — идемпотентность (топ-10%), слабее — индексы БД. Грейд подтверждён 2 мес. назад. Призёр ФСП».
- Фильтры уточняют запрос **без потери подборки**: заказ сохраняется, фильтры накладываются поверх.
- **Приглашение:** описание, зарплата «от–до» в ₽ (обязательно), компания, способ связи. Статусы: отправлено → просмотрено → принято / отклонено. Контакты раскрываются только после принятия.

8.2 «Слепая дегустация»

1. Работодатель публикует для категории **регулярную задачу**, собранную по его заказу. У каждого кандидата при этом своя начинка.
2. Кандидаты решают анонимно. Результат идёт в рейтинг на вертеле, мясо снова горячее.
3. Работодатель **пробует решения через «окошко ларька»**: видит решение, разбор по ингредиентам и рейтинг, но **не видит, кто автор**. Понравилось — отправляет приглашение с вилкой зарплаты.
4. Если «заказали» несколько компаний, кандидат видит **«Очередь из 3 компаний: 280–320, 300–350, 330–380 тыс. ₽»** и выбирает сам. Сила переходит к кандидату, компании конкурируют условиями.
5. Слепой отбор убирает предвзятость по возрасту, вузу, полу и инвалидности. Это прямая связь с инклюзивностью платформы.

8.2a «Свежая начинка» — адресная задача от системы

ТЗ описывает регулярную задачу как то, что **система периодически предлагает кандидату**. Поэтому, кроме задач от работодателей, шаурматч_ сам раз в 7 дней адресно подбирает каждому кандидату с подтверждённой категорией короткую задачу: 1. **Какой ингредиент.** Тот, которому подтверждение нужнее всего: рейтинг «остыл» (давно не было практики, RD вырос), ингредиент ещё ни разу не проверялся или это самое слабое место относительно грейда. Кандидат видит причину: «Рейтинг „REST-контракты" остыл: 220 дн. без практики — работодатели видят его ниже». 2. **Какая задача.** Шаблон и начинка, которые покрывают этот ингредиент, со сложностью чуть ниже уровня кандидата (≈65% успеха): так ответ максимально информативен и не демотивирует. Уже решённые шаблоны не повторяются. 3. **Оценка**

автоматическая. Вариант уникален, эталон получен исполнением кода, результат — «матч» на вертеле. Задача приходит при заходе в «Задачи» и письмом (периодический обход раз в 6 часов). Работодатель видит в карточке кандидата, что тот поддерживает форму («практика шаурматч_»).

Код: `engine/app/practice.py` (выбор), `backend/src/services/practice.ts` (когда и кому), миграция `003_system_tasks.sql`.

8.3 «Честный ценник» (защита от фиктивных вакансий — «будет плюсом» в ТЗ)

- **Талоны на приглашения:** у компании ограниченное число приглашений в неделю. Лимит растёт с репутацией. Талон возвращается, если кандидат ответил. Массовая рассылка невыгодна, и это лечит болезнь, которая сгубила обратные площадки.
- **Ценник не врёт:** после собеседования кандидат одним кликом подтверждает, совпала ли зарплата с заявленной вилкой. Если нет, у компании падает репутация, урезаются талоны, и бейдж «Честный ценник» снимается.
- **Признаки фиктивности:**
- вакансия висит месяцами без найма;
- все приглашения с одинаковым текстом;
- вилка сильно выше рынка (приманка);
- много приглашений, ни одного найма.
- **Интеграция с внутренними системами подбора компаний (ATS) — концепция:** приглашения и статусы отправляются вебхуками и выгружаются в Хантфлоу, Potok, E-Staff.

8.4 Что видит работодатель и как читать выдачу

- **Кольцо «совпадение»** — P(подойдёт): вероятность, что кандидат пройдёт собеседование по этому заказу (ранжировщик «Маринада»). **Порядок** — шанс найма = P(подойдёт) × P(примет приглашение с вашей вилкой); вторая часть показана рядом («примет ≈72%» и что мешает). Место в выдаче и процент считаются одной моделью и не спорят друг с другом.
- **ФИО, опыт и места работы видны по умолчанию** (ответ организаторов VS.4: закрываются только контакты). Кандидат может скрыть имя и места работы — тогда он «Кандидат #код». Анонимность остаётся принципом «Слепой дегустации».
- **Стек: со слов и подтверждённый.** «✓ Python» — кандидат решил не меньше двух заданий на этом языке сложностью не ниже уровня своего грейда. Простая доля верных не годится: адаптивный тест держит её около 60% у всех. Остальной стек — со слов кандидата; это фильтр, а не часть категории.
- **Резюме — подсказка, а не категория.** Разбор PDF извлекает ФИО, контакты, стек, роли, места работы с периодами, стаж и софт-скиллы; собственная языковая модель шаурматч_ подсказывает специализацию и навыки для предзаполнения опроса. Категорию и место в выдаче определяет только тест.

8.5 Как распространить метод на дизайнеров (концепт)

Организаторы допускают дизайнеров в справочник, если есть объективная проверка (WA.2). ШАУРМА переносится без изменения ядра: - **Лаваш** — макет интерфейса (форма, карточка, таблица), сгенерированный кодом по шаблону. - **Начинка** — операторы-нарушения гайдлайна с известной сложностью η: контраст ниже 4.5:1, сломанная сетка на 4 px, неверный порядок фокуса, «висящий» заголовок, разные радиусы у однотипных элементов. - **Задание** — «найди нарушение»: варианты ответа — области макета. Эталон известен, потому что нарушение внесено генератором; проверка контраста и сетки вычисляется, а не оценивается на глаз. - **Ингредиенты** — доступность, сетка и типографика, визуальная иерархия, консистентность дизайн-системы. Рецепт вакансии «продуктовый дизайнер B2B» раскладывается на них так же, как для разработчиков.

9. A — Антислив: фирменный соус

- **Канарейки:** в каждом варианте уникальные метки — константы, имена, порядок данных, иногда «ловушка», правильная только для этого варианта. Если в ответе кандидата Б есть ответ к варианту кандидата А, это **доказательство списывания** и источник утечки.
- **Статистика соответствия ответов модели (person-fit):** трудные задания решены, лёгкие провалены, аномально быстрые ответы. Такая попытка отправляется на перепроверку.
- **Проверка пониманием:** при подозрении кандидат получает 2–3 уточняющих вопроса по своему решению (ИИ-интервью).

---

10. Соответствие ТЗ ФСП

Требование ТЗ	Как закрыто
Категория через опрос → самовыбор грейда → тест	§5.2
Задания устойчивы к распространению	§4 открытый банк + §6 уникальная начинка + §9 канарейки
Сопоставимая сложность при уникальных заданиях	§6 LLTM + эталон из исполнения кода
Показать, как из вакансий формируются задания	§5.1 «вакансия → рецепт»
Грейд не понижается принудительно, смена ограничена по времени	§5.2, 90 дней
Регулярные короткие задачи от работодателя	§8.2 «Слепая дегустация» + §7 вертел
Ранжирование внутри категории, приоритет ФСП	§7 μ − 2·RD + бонус ФСП
Выдача по категориям, а не по резюме, с объяснением	§8.1
Уточнение без потери подборки	§8.1
Обработка кандидата без ФСП	§7
Механика контакта, отличная от отклика	§8.1–8.2
Защита от фиктивных вакансий, ATS (плюс)	§8.3
Собственная процедура валидации	§11

11. Валидация (обязательный раздел сдачи)

Готовых данных нет, поэтому строим **синтетический мир с известной истиной**:

1. **2 000 синтетических кандидатов** с заданными истинными уровнями по ингредиентам, специализацией, стеком и (у 40%) историей ФСП. Ответы моделируются по моделям LLTM и IRT, а ИИ генерирует правдоподобные резюме.
2. **Тест:** - восстановление грейда: точность и Cohen's κ между истинным и присвоенным грейдом; - **тест-ретест** на разных вариантах: согласованность грейда; - восстановление профиля ингредиентов: корреляция с истиной; - дискриминативность заданий: точечно-бисериальная корреляция, информация задания; - **симуляция слива:** 30% кандидатов знают весь банк и ответы чужих вариантов. Сравниваем завышение грейда (фиксированный тест против ШАУРМЫ) и долю пойманных канарейками.
3. **Подбор:** 60 описаний вакансий и размеченные пары «вакансия — кандидат» (по истине, часть проверяется вручную). Метрики **Precision@10 и nDCG@10** в сравнении с поиском по ключевым словам (BM25) и поиском по эмбеддингам резюме.
4. **Главный слайд питча:** «Слили весь банк задач: обычный тест завышает грейд, ШАУРМА — нет». Цифры появятся после прогона симуляции.

11a. «Маринад»: метод учится на собственных данных

Экспертные числа метода (сложности b0 и η, дискриминация, словарь «слово → навык», веса подбора) заменяются моделями, которые дообучаются на действиях пользователей. Ответы на задания калибруют тест (LLTM + 3PL, MML-EM со страховочным шлюзом). Детектор «чёрствого лаваша» замечает, что приём начинки «утёк». Правки рецептов учат собственную языковую модель. Ответы на приглашения и итоги собеседований учат двусторонний ранжировщик P(подойдёт) × P(примет).

В симуляции «года жизни платформы» завышенных грейдов стало на 30% меньше, наймов на одно приглашение — на 37% больше, а «утёкший» приём пойман за 123 ответа без ложных тревог. Подробно: МАРИНАД.md, страница `/marinade`.

12. Объём MVP

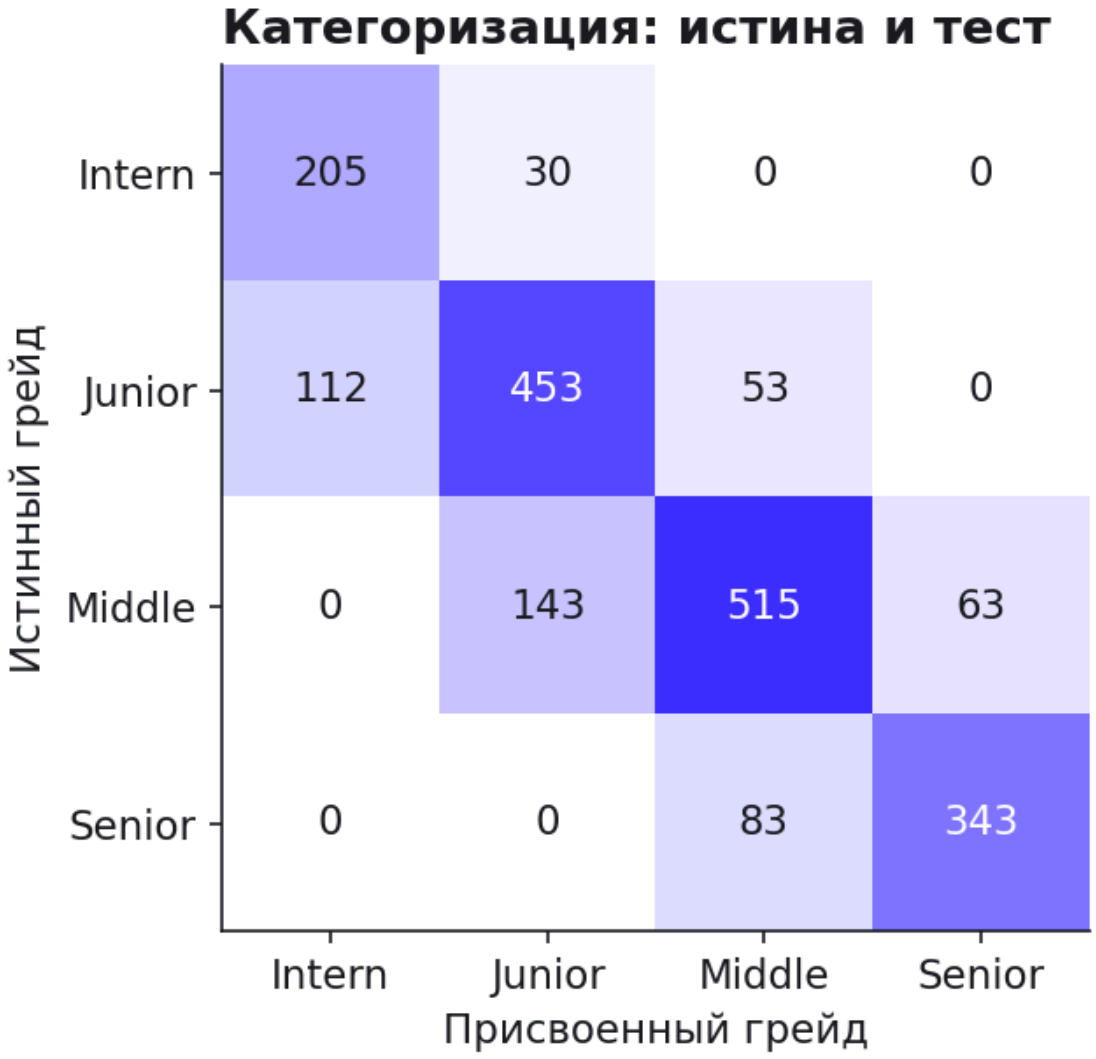
- **Обязательно:** меню ингредиентов (3 специализации × ~8 ингредиентов), 25–40 шаблонов, 10–15 операторов мутации, адаптивный тест, Glicko-2, заказ → рецепт, подборка с объяснением, приглашения, канарейки, скрипт валидации.
- **Плюсом:** «Слепая дегустация», талоны и «Честный ценник», отклики на вакансии, мобильная версия.
- **Концепт для питча:** ИИ-агенты, ИИ-интервью, интеграция с ATS, найм команд.

4. Валидация метода: процедура и результаты

Синтетическая популяция: **2000 кандидатов** (seed 42) с известными истинными навыками по каждому ингредиенту; самооценка грейда с типичным завышением (30% завышают на уровень). Ответы моделируются по 3PL-модели от истинных навыков ингредиентов задания (многомерная истина при одномерном тесте — проверка устойчивости). Скрипт: `engine/validation/simulate.py`, сырые цифры: `metrics.json`.

1. Категоризация

Метрика	Значение
Точное совпадение грейда с истиной	**76%**
Совпадение ±1 уровень	**100%**
Квадратично-взвешенная к Коэна	**0.87**
Доля завышенных грейдов	7%
Среднее число попыток (включая тест на уровень ниже)	1.31
Восстановление профиля ингредиентов (ρ Спирмена, среднее)	**0.671**



2. Стабильность (тест-ретест на разных уникальных вариантах)

Метрика	Значение
Совпадение грейда при повторном прохождении	**75%**
к (квадратичная) тест-ретест	**0.864**

3. Дискриминативность заданий

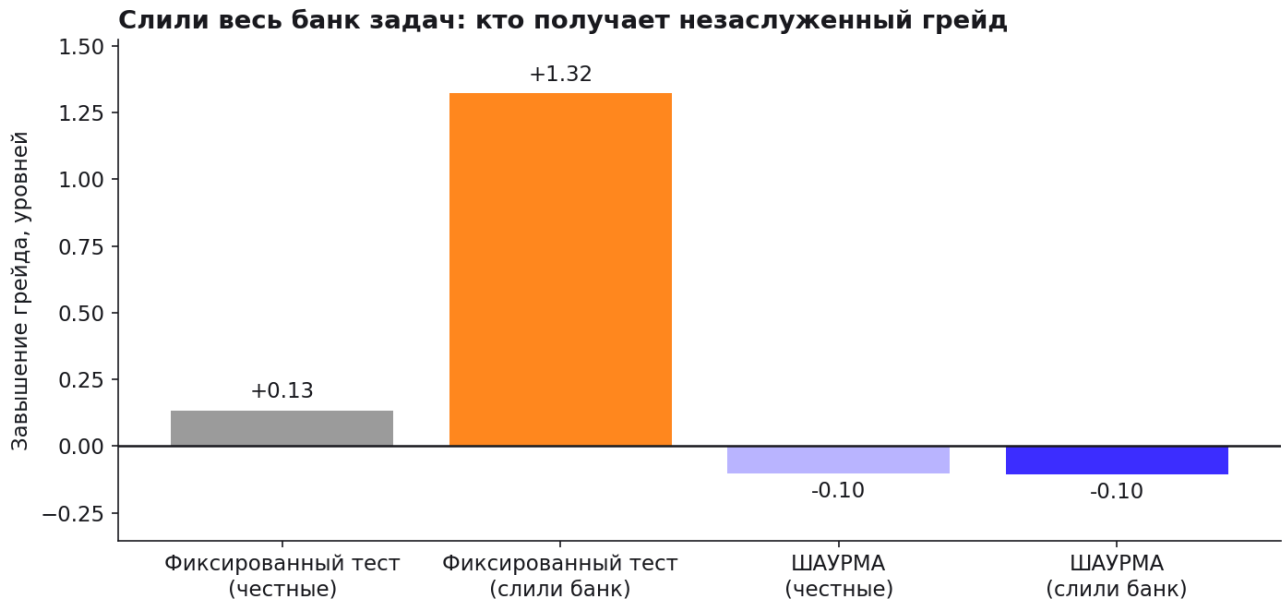
Средняя точечно-бисериальная корреляция «верно ↔ истинный уровень»: **0.341** (ориентир психометрики: > 0.2 — приемлемо, > 0.3 — хорошо). Максимальная доля показов одного шаблона: 4.3%.

4. Устойчивость к сливу

Сценарий: кандидаты знают **весь** банк задач и ответы на чужие варианты.

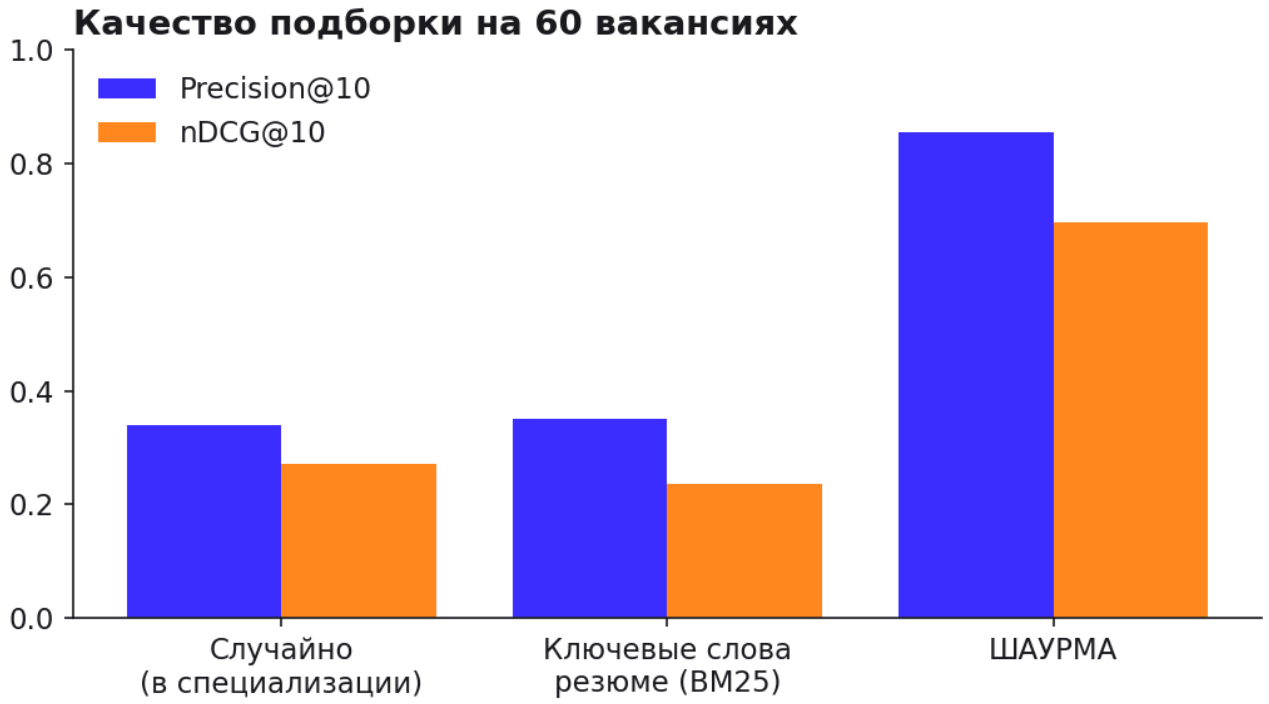
	Завышение грейда
Фиксированный тест, честные	+0.13
Фиксированный тест, слили банк	**+1.32**
ШАУРМА, честные	-0.10
ШАУРМА, слили банк	**-0.10**

Канарейки поймали **92%** списывающих при ложных срабатываниях на честных **1.3%** (порог — 2 совпадения с ответами чужих вариантов; подозрение ведёт не к бану, а к уточняющим вопросам).



5. Подбор (60 вакансий, истинная релевантность известна)

Подход	Precision@10	nDCG@10
Случайно внутри специализации	0.34	0.272
Ключевые слова резюме (BM25)	0.35	0.235
ШАУРМА (категория + ингредиенты + рейтинг)	**0.855**	**0.697**



Ограничения

- Мир синтетический: абсолютные значения зависят от допущений модели ответов. На реальных данных сложности η операторов и дискриминации заданий калибруются заново (LLTM-регрессия по накопленным ответам).
- «Приукрашивание» резюме (30% кандидатов) — допущение; именно от него страдает BM25.

5. «Маринад»: обучение на данных платформы

> Шаурма вкуснее, чем дольше маринуется. шаурматч_ точнее, чем больше им пользуются.

В методе «ШАУРМА» есть числа, которые команда задала «на глаз»: сложность каждого шаблона и каждой начинки, общая дискриминация заданий, словарь «слово → навык», веса формулы подбора. «Маринад» заменяет их моделями, которые дообучаются на действиях, уже существующих в продукте. Отдельные разметчики, внешние нейросети и чужие данные не нужны.

Действие в продукте	Что дообучается	Модель	Код
Кандидат отвечает на задание	Сложность шаблонов b0, начинок η, дискриминация a	LLTM + 3PL, MML-EM с априорными распределениями вокруг экспертных значений	`engine/app/learn/calibration.py`
Те же ответы, по времени	Детектор «чёрствого лаваша»: начинка внезапно стала легче	CUSUM по изменению остатков	`engine/app/learn/drift.py`
Работодатель правит рецепт ползунками	Разбор текста потребности в навыки	Своя языковая модель: символьные n-граммы и линейные головы	`engine/app/learn/textmodel.py`
Кандидат принимает или отклоняет приглашение, работодатель отмечает «наняли / не подошёл»	Порядок выдачи подборки	Двусторонний ранжировщик P(подойдёт) × P(примет)	`engine/app/learn/ranker.py`

Посмотреть вживую: страница `/marinade` (интерактивный «год в перемотке», «попробуйте сами», живой паспорт моделей). Полный отчёт с графиками: training/report.md.

Результаты (симуляция «года жизни платформы»)

Мир синтетический, но с известной истиной. Эксперт ошибся в сложностях заданий так же, как ошибаются эксперты в психометрической литературе. У людей есть скрытые навыки и скрытые правила принятия решений. За 12 месяцев приходят 3600 кандидатов и 360 вакансий,

накапливается 86 тыс. ответов. В 7-м месяце завирусился разбор начинки «Бинарный поиск: строгое/нестрогое сравнение». Сравнение «до/после» сделано на одних и тех же 2000 новых кандидатах и 150 вакансиях.

	До (эксперт / ручная формула)	После «Маринада»
Ошибка сложности начинок η, RMSE (логиты)	0,307	**0,130**
Ошибка сложности шаблонов b0, RMSE	0,361	**0,138**
Ошибка дискриминации, RMSE log a	0,268	**0,098**
Корреляция оценок η с истиной	0,60	**0,95**
Доля незаслуженно завышенных грейдов	7,4%	**5,2%** (−30%)
Грейд определён точно	77,3%	**79,1%**
Наймов на 10 приглашений	1,90	**2,61** (+37%)
Приглашений принято	49%	**65%**
«Утёкший» приём начинки обнаружен	—	**через 123 ответа** (оценка: легче на 0,58 логита), 0 ложных тревог за год

Языковая модель проверена на 40 описаниях потребностей, написанных вручную отдельно от обучающего корпуса. В них намеренно есть формулировки, которых нет в словаре: «двойные списания», «одно место продаётся двум людям», «трёхзначная логика».

	Словарь правил	Модель шаурматч_
Специализация	90%	**98%**
Грейд	83%	83%
Заказанные навыки в топ-3 рецепта	50%	**68%**

На правках работодателей модель доучивается скромно, и мы этого не скрываем. После 30 правок (перекрёстная проверка по 4 фолдам) грейд вырос с 83% до 88%, навыки в топ-3 — с 67% до 68%.

Пересчитать всё: `cd engine && python -m training.marinate --bake` (≈4 минуты, seed 7). Банк — 34 шаблона и 123 набора начинок.

Как это работает в продукте

1. **Калибровка теста.** Каждые `CALIBRATE_EVERY` (по умолчанию 300) новых ответов backend (`backend/src/services/learning.ts`) отправляет все ответы в движок (`POST /learn/calibrate`). - **Страховочный шлюз.** Модель обучается на 80% кандидатов. Новая

версия включается, только если на оставшихся 20% она объясняет ответы лучше текущей и лучше экспертной (маргинальное правдоподобие). - **Априорные распределения** держат параметры у экспертных значений, пока данных мало. Чем больше ответов, тем сильнее параметр сдвигается к тому, что показали люди. - **Шкала** привязана к экспертной (mean-mean linking), поэтому пороги грейдов не уплывают. - **В образ калибровка не запекается.** Она учится только на живых ответах, иначе тест выучил бы сложности вымышленного мира.

2. **Детектор «чёрствого лаваша»** работает перед каждой калибровкой. - Остатки ответов считаются относительно текущей модели. Тревога поднимается, когда остатки начинки растут по сравнению с её собственной историей (k = 0,3, h = 15, разогрев 150 ответов). - Пороги подобраны так, чтобы за год симуляции на ~120 начинках не было ни одной ложной тревоги (максимум CUSUM на честных начинках — 12,9 при пороге 15). - Величина изменения оценивается в момент тревоги: свежее окно ответов против истории до него. - Тревога обнуляет историю начинки: следующая калибровка выучивает её новую сложность.
3. **Ранжировщик** стартует «тёплым»: веса выучены на симуляции. Каждые `RANKER_EVERY` (20) новых ответов на приглашения голова «Примет» дообучается с L2-притяжением к стартовым весам; голова «Подойдёт» — только когда накопится не меньше 50 итогов собеседований (на десятке примеров веса становятся шумом). Если обучен ранжировщик, выдача сортируется по P(найм), а в объяснении появляется строка «вероятно примет приглашение: 72% (мешает: вилка ниже ожиданий)». У работодателя в подборке — оценка «примет ≈N%», в приглашениях — кнопки «Наняли / Не подошёл».
4. **Языковая модель** запечена в образ (8000 сгенерированных текстов + 40 ручных) и переобучается после каждой правки рецепта. Правка весит в обучении в 4 раза больше сгенерированного текста.

Все версии пишутся в журнал `model_versions` (что обучили, на скольких примерах, включили ли и почему). Модели — открытые JSON-файлы: `engine/app/models/` (запечённые) и том `models` (дообученные). Откат к запечённой или экспертной версии: `POST /learn/reset {"name": "calibration" | "ranker" | "textmodel"}`.

API

Метод	Назначение
`GET /api/v1/public/learning`	Паспорт моделей: версии, накопленные сигналы, детектор, журнал
`POST /api/v1/public/learning/recipe`	«Попробуйте сами»: текст → рецепт словарём и моделью (лимит 30/мин)
`POST /api/v1/employer/invitations/{id}/outcome`	Итог собеседования: `hired` / `not_fit`
движок `POST /learn/calibrate`, `/learn/drift`, `/learn/ranker`, `/learn/textmodel`, `/learn/reset`, `GET /learn/status`	Обучение (внутренняя сеть)

Ограничения

- Цифры «года» получены на синтетическом мире: проверен механизм, а не величина эффекта на реальном рынке.
- Ручная выборка для языковой модели маленькая (40 текстов), погрешность около ±7 п. п.
- Ранжировщик учится на приглашениях, которые сам же предложил. Против замыкания на собственном топе в симуляции одно приглашение из пяти — «тёмная лошадка» из топ-30. В продукте выбор остаётся за работодателем.
- Ранжировщик выучил положительный вес у «неопределённости рейтинга». Это поправка к консервативной оценке μ − 2·RD, которая занижает кандидатов с малым числом заданий, а не «любовь к неизвестности».

6. «Маринад»: отчёт симуляции

Шаурма вкуснее, чем дольше маринуется. шаурматч_ точнее, чем больше им пользуются: три модели дообучаются на том, что люди и так делают в продукте. Никаких отдельных разметчиков и внешних API.

Действие в продукте	Чему учит	Модель
Кандидат отвечает на задание теста	Сложность каждой начинки и шаблона, их различающую силу	Калибровка LLTM + 3PL (MML-EM с априорными распределениями вокруг экспертных значений)
Работодатель правит рецепт ползунками	Как текст потребности превращается в навыки	Собственная языковая модель «текст → рецепт» (символьные n-граммы, линейные головы)
Кандидат принимает или отклоняет приглашение, работодатель отмечает найм	Кого приглашать, чтобы найм состоялся	Двусторонний ранжировщик: P(подойдёт) × P(примет)

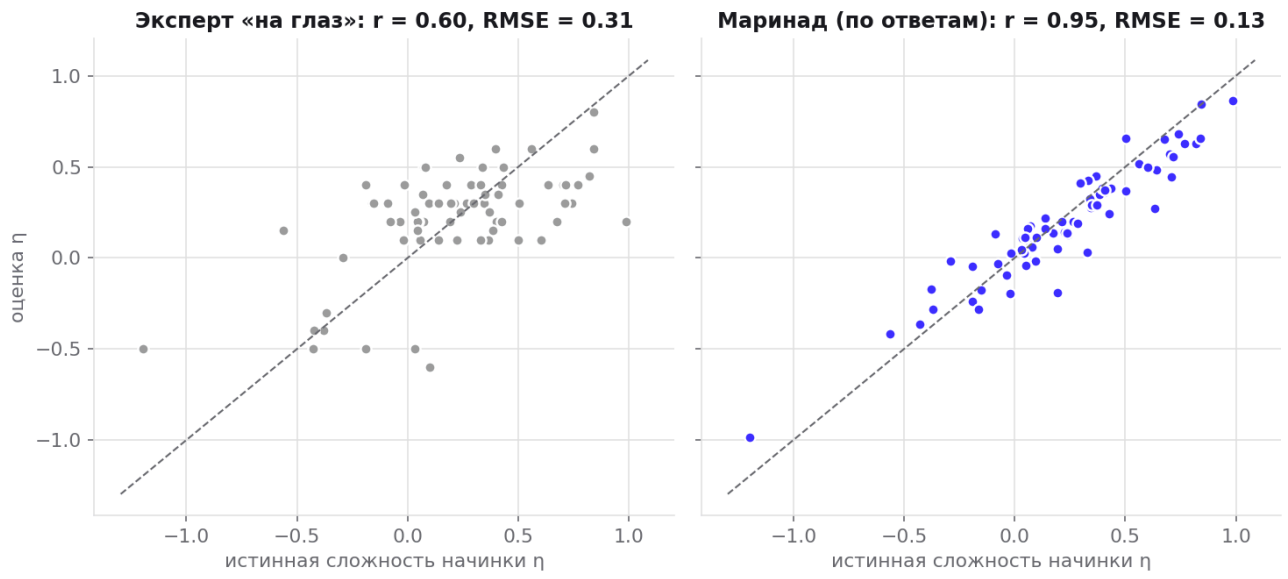
Код: `engine/app/learn/`, симуляция: `engine/training/marinate.py`, сырые цифры: `metrics.json`. Все модели — открытые JSON-файлы с весами, без pickle и обфускации.



1. Год жизни платформы в ускоренной перемотке

Синтетический мир с известной истиной: эксперт ошибся в сложностях заданий примерно так же, как ошибаются эксперты в психометрической литературе, у кандидатов есть скрытые навыки по каждому ингредиенту, а решения «принять приглашение» и «нанять» они принимают по скрытым правилам (деньги, формат, переквалификация, репутация компании, истинное совпадение навыков). Каждый месяц приходят 300 кандидатов и 30 вакансий, а в конце месяца модели дообучаются.

	Месяц 1	Месяц 12
Накоплено ответов на задания	7,481	85,864
Ошибка сложности начинок η, RMSE (эксперт: 0.312)	0.205	**0.13**
Ошибка базовой сложности шаблонов, RMSE (эксперт: 0.361)	0.179	**0.138**
Ошибка дискриминации, RMSE log a (эксперт: 0.268)	0.176	**0.098**
Наймов на 10 приглашений	1.556	**2.393**
Приглашений принято	39%	**63%**

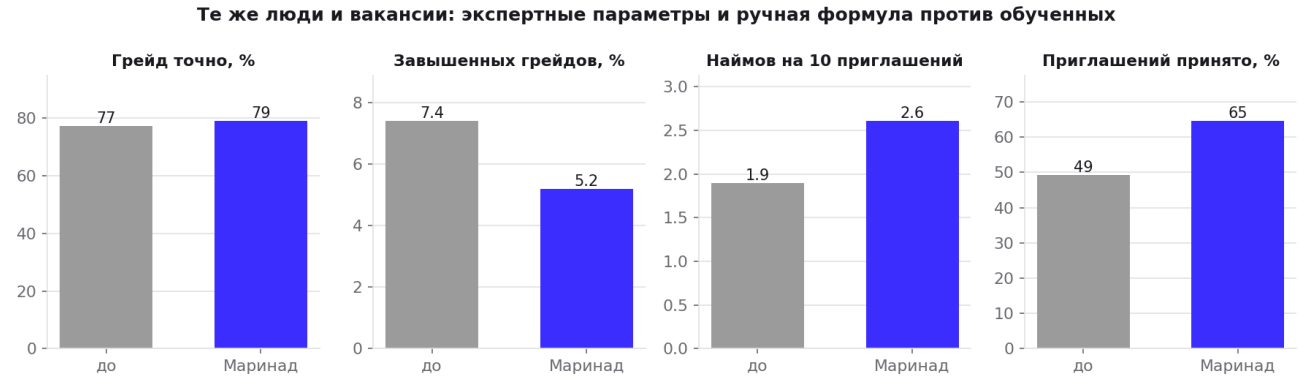


2. До и после на одних и тех же людях

Свежая когорта из 2000 кандидатов проходит тест дважды на одних и тех же случайных числах: с экспертными параметрами и с выученными. 150 новых вакансий ранжируются ручной формулой ШАУРМЫ и обученным ранжировщиком.

Метрика	До (эксперт / формула)	После (Маринад)
Грейд определён точно	77.3%	**79.1%**
к Коэна (квадратичная)	0.875	**0.88**
Доля завышенных грейдов	7.4%	**5.2%**

Метрика	До (эксперт / формула)	После (Маринад)
Наймов на 10 приглашений	1.899	**2.609**
Приглашений принято	49%	**65%**



3. Детектор «чёрствого лаваша»

Банк шаблонов ШАУРМЫ открыт, и ответы не утекают: варианты уникальны. Но может «утечь приём»: разбор трюка начинки завирусился, и задание перестаёт различать сильных и слабых. В 7-м месяце так случилось с начинкой «Бинарный поиск: границы: строгое/нестрогое сравнение» (`py_binary_search:upper`): 70% новых кандидатов видели разбор.

Для каждого ответа считается остаток «ответ минус предсказание текущей модели». У любой начинки есть постоянное небольшое несоответствие модели, поэтому детектор ищет не отклонение от нуля, а **изменение**: после разогрева (150 ответов) он копит CUSUM по отклонению остатков от собственной прошлой нормы начинки (k = 0.3, h = 15.0). Детектор поднял тревогу в 7-м месяце, через **123 ответа** на эту начинку после начала разбора, и оценил, что она стала легче на **0.58 логита**. История этой начинки обнулилась, и следующая калибровка выучила её новую сложность. Сложность этой начинки: эксперт +0.35, истинная до разбора -0.19, после разбора в среднем по кандидатам -1.10; «Маринад» к концу года выучил **-0.87**. Ложных тревог на других начинках за год: 0.

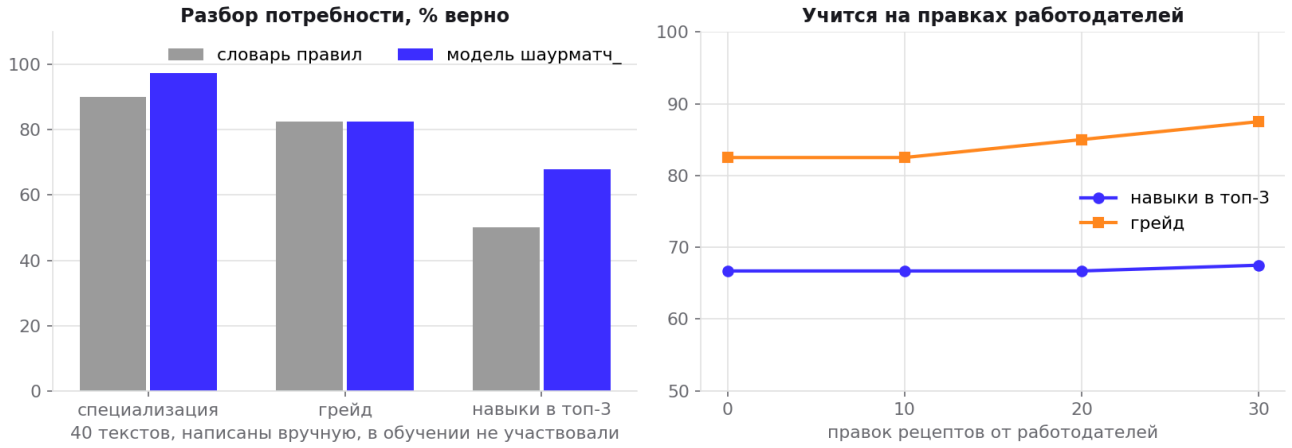
4. Своя языковая модель вместо словаря

Корпус — 8,000 сгенерированных описаний потребностей (банки формулировок по навыкам, грейдам и стекам; парсить чужие вакансии ТЗ запрещает). Проверка — на **40 текстах, написанных вручную отдельно** и генератором не порождаемых: в них много формулировок, которых нет в словаре («двойные списания», «одно место продаётся двум людям», «трёхзначная логика»).

	Словарь правил	Модель шаурматч_
Специализация	90%	**98%**
Грейд	82%	**82%**

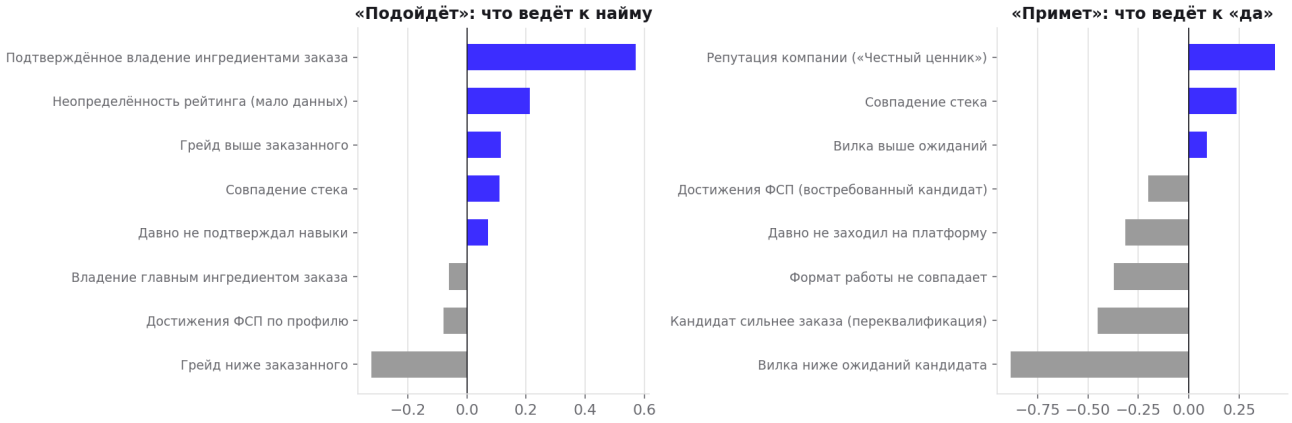
	Словарь правил	Модель шаурматч_
Заказанные навыки в топ-3 рецепта	50%	**68%**

Обучение на правках (4 фолда: модель видит k вручную размеченных текстов как «правки работодателей» и проверяется на остальных): навыки в топ-3 67% → 67% → 67% → 68%, грейд 82% → 82% → 85% → 88% при k = 0, 10, 20, 30.



5. Чему научился подбор

Веса обеих голов открыты и названы по-русски, поэтому подбор остаётся объяснимым: работодатель видит не только «почему кандидат подходит», но и «примет ли он ваше приглашение».



Как это работает в продукте

- **Калибровка теста** учится только на живых ответах: в образ она не запекается, иначе тест выучил бы сложности вымышленного мира. Пока ответов мало, априорное распределение держит параметры у экспертных значений. Пересчёт запускается автоматически каждые 300 новых ответов и пишется в журнал версий (`model_versions`).

- **Ранжировщик** стартует с весов, выученных на симуляции («тёплый старт»), и дообучается на реальных ответах на приглашения с L2-притяжением к стартовым весам.
- **Языковая модель** запечена в образ (корпус + ручные тексты) и переобучается, когда работодатель правит рецепт.
- **Откат** к экспертным параметрам — одна команда; каждая версия модели хранится с метриками.

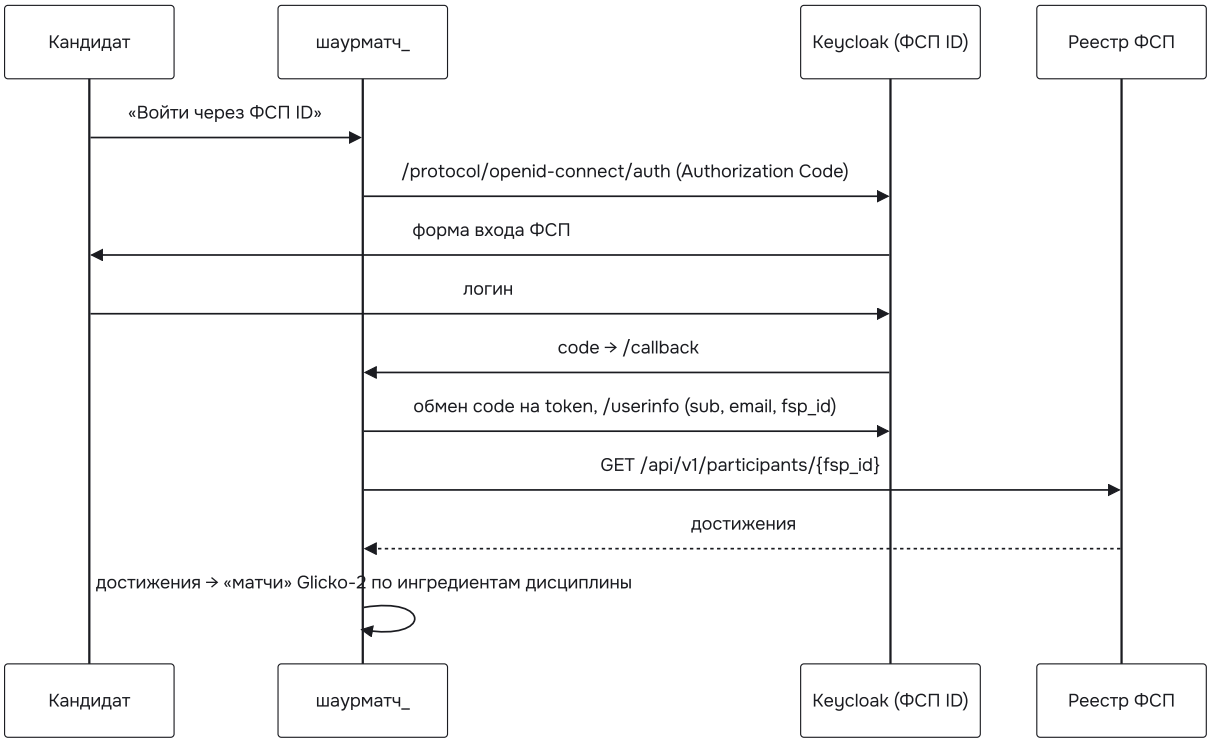
Ограничения

- Мир синтетический: абсолютные цифры зависят от допущений о поведении людей. Механизм проверен, величина эффекта на реальном рынке будет другой.
- Ручная выборка для языковой модели маленькая (40 текстов), поэтому погрешность около ±7 п. п.
- Ранжировщик учится на приглашениях, которые сам же и предложил. Чтобы не замкнуться на собственном топе, одно из пяти приглашений в симуляции — «тёмная лошадка» из топ-30.

7. Интеграция с реестром ФСП

ТЗ (п. 4): API и структура данных ФСП в рамках хакатона не предоставляются, команда сама предлагает, какие сведения получать и как их показывать. Ниже — наше предложение. Оно реализовано в мок-сервисе `fsp-mock/` и в клиенте `backend/src/services/fsp.ts`.

1. Идентификация: ФСП ID на Keycloak



- Реализовано в `backend/src/routes/auth.ts`. Включается переменными `KEYCLOAK_*`; demo-realm лежит в `keycloak/realm-fsp.json`, запуск — `docker-compose.keycloak.yml`.
- Без Keycloak кандидат вводит ID участника вручную: `POST /api/v1/candidate/fsp/link`. Если ID уже привязан к другому профилю, возвращается 409. В промышленной версии привязку нужно подтверждать входом через ФСП ID.

2. Предлагаемая структура данных реестра

`GET /api/v1/participants/{fsp_id}`

```
{
  "fsp_id": "FSP-2025-0001",
  "full_name": "Алексей Смирнов",
  "region": "Москва",
  "achievements": [{
    "event": "Чемпионат России по спортивному программированию 2025",
    "date": "2025-11-20",
    "discipline": "product",
```

```
    "discipline_title": "Продуктовое программирование",
    "level": "national",
    "level_title": "Всероссийский",
    "place": 2,
    "participants": 148,
    "team": "NullPointer",
    "role": "капитан, backend",
    "rank_title": "КМС"
  }]
}
```

Справочники: `GET /api/v1/disciplines`, `GET /api/v1/levels`. Дисциплины соответствуют видам спортивного программирования: алгоритмическое, продуктовое, информационная безопасность, робототехника, беспилотные авиасистемы.

3. Как достижения влияют на подбор

1. Каждое достижение превращается в **«матчи» Glicko-2** по ингредиентам дисциплины (`engine/app/glicko.py → DISCIPLINE_INGREDIENTS`). Например, продуктовое программирование даёт REST-контракты, отладку, изменяемое состояние и SQL-агрегации.
2. «Соперник» — рейтинг уровня соревнования: городской 1450 … международный 1850. Результат: призовое место или топ-20% — победа, топ-50% — ничья, остальные — ничья с соперником уровнем ниже.
3. Внутри категории это **повышает силу профиля**: `μ - 2·RD` плюс бонус за призовые места. В подборке достижения ФСП дают небольшой бонус к совпадению (до +15%) и попадают в объяснение («Достижения ФСП: 3, призовых 2»).
4. **Нет истории ФСП — нет штрафа.** Достижения только добавляют информацию. Профиль подтверждается тестами, а в интерфейсе выводится нейтральная пометка «квалификация подтверждена тестами платформы». Фильтр «только с ФСП» работодатель включает сам.
5. **Найм команд (концепция).** Поле `team` позволяет находить кандидатов, выступавших вместе. Это основа будущего группового приглашения.

4. Отказоустойчивость

Если реестр недоступен, запрос возвращает 503 с понятным сообщением, а остальная платформа работает. Достижения кэшируются в профиле кандидата (`fsp_achievements`) и обновляются при повторной привязке. В продакшене — периодической синхронизацией по `fsp_linked_at`.

8. Описание API

Полная спецификация OpenAPI 3: `docs/openapi.json`, интерактивно — Swagger UI на `/api/docs`. Базовый путь: `/api/v1`. Методов: 65. Авторизация — httpOnly-cookie или `Authorization: Bearer`; ошибки — JSON `{error, message, details?}`.

Аутентификация

Метод	Путь	Назначение	Доступ	Коды
`POST`	`/auth/register`	Регистрация по email (с подтверждением адреса)	публично	200, 400, 404
`POST`	`/auth/verify`	Подтверждение email по токену из письма	публично	200, 400, 404
`POST`	`/auth/login`	Вход по email и паролю	публично	200, 400, 404
`POST`	`/auth/resend`	Повторно отправить письмо подтверждения	публично	200, 400, 404
`POST`	`/auth/logout`	Выход	публично	200, 400, 404
`GET`	`/auth/me`	Текущий пользователь (null — не авторизован)	публично	200, 400, 404
`GET`	`/auth/providers`	Доступные способы входа (email, ФСП ID через Keycloak)	публично	200, 400, 404

Кандидат

Метод	Путь	Назначение	Доступ	Коды
`GET`	`/candidate/profile`	Мой профиль	по входу	200, 400, 401, 403, 404
`PUT`	`/candidate/profile`	Заполнить или обновить профиль вручную	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/resume`	Загрузка PDF-резюме с авторазпознаванием (ФИО, контакты, стек, грейд, роли, стаж, софт-скиллы) Распознанные данные предзаполняют профиль и опрос. Категорию резюме не определяет — только тест (требование ТЗ).	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/survey`	Опрос по отрасли и специализации + выбор предполагаемого грейда	по входу	200, 400, 401, 403, 404

`PUT`	`/candidate/privacy`	Приватность и согласия (152-ФЗ)	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/fsp/link`	Связать профиль с ID участника ФСП и подтянуть достижения Достижения ФСП засчитываются как внешние подтверждённые «матчи» по ингредиентам дисциплины (рейтинг Glicko-2).	по входу	200, 400, 401, 403, 404
`DELETE`	`/candidate/fsp/link`	Отвязать ID ФСП	по входу	200, 400, 401, 403, 404
`GET`	`/candidate/profile/pdf`	Стандартизированный PDF-профиль (паспорт навыков)	по входу	200, 400, 401, 403, 404
`GET`	`/candidate/grade-history`	История опроса и тестов на грейд	по входу	200, 400, 401, 403, 404
`GET`	`/candidate/market`	Моя категория на рынке: сколько компаний ищут, медиана, моё место	по входу	200, 400, 401, 403, 404

Справочники

Метод	Путь	Назначение	Доступ	Коды
`GET`	`/dict`	Специализации, грейды, ингредиенты, стек, параметры теста	публично	200, 400, 404

Публичное

Метод	Путь	Назначение	Доступ	Коды
`GET`	`/public/stats`	Счётчики для лендинга	публично	200, 400, 404
`GET`	`/public/market`	Открытая карта рынка: кандидаты и медианные ожидания по категориям	публично	200, 400, 404

Работодатель

Метод	Путь	Назначение	Доступ	Коды
`GET`	`/employer/company`	Профиль компании	по входу	200, 400, 401, 403, 404

`PUT`	`/employer/company`	Обновить профиль компании	по входу	200, 400, 401, 403, 404
`POST`	`/employer/needs/parse`	Разобрать описание потребности в рецепт (без сохранения)	по входу	200, 400, 401, 403, 404
`POST`	`/employer/needs`	Описать потребность: какие специалисты нужны, чем занимается команда, стек, грейд	по входу	200, 400, 401, 403, 404
`GET`	`/employer/needs`	Мои потребности	по входу	200, 400, 401, 403, 404
`PUT`	`/employer/needs/{id}`	Поправить рецепт (веса ингредиентов, грейд, стек)	по входу	200, 400, 401, 403, 404
`GET`	`/employer/needs/{id}/selection`	Подборка по потребности: рекомендованные категории + ранжированные кандидаты с объяснением Фильтры применяются поверх сохранённого рецепта — подборка не теряется при уточнении запроса.	по входу	200, 400, 401, 403, 404
`GET`	`/employer/market`	Карта рынка: категории (специализация × грейд), число кандидатов, медиана ожиданий	по входу	200, 400, 401, 403, 404
`GET`	`/employer/candidates`	Банк кандидатов: фильтры по специализации, грейду, категории, стеку, достижениям ФСП; ранжирование внутри категории Внутри категории кандидаты ранжируются по силе подтверждённого профиля: консервативный рейтинг μ − 2·RD по ингредиентам + призовые места ФСП.	по входу	200, 400, 401, 403, 404

`GET`	`/employer/candidates/{id}`	Карточка кандидата (контакты — только после принятия приглашения или отклика)	по входу	200, 400, 401, 403, 404
`GET`	`/employer/stats`	Воронка приглашений и причины отказов	по входу	200, 400, 401, 403, 404

Приглашения

Метод	Путь	Назначение	Доступ	Коды
`POST`	`/employer/invitations`	Пригласить кандидата: описание, зарплата «от–до» (обязательно), способ связи Вакансия не обязательна. Расходует «талон» (антиспам «Честного ценника»).	по входу	200, 400, 401, 403, 404
`GET`	`/employer/invitations`	Мои приглашения и их статусы	по входу	200, 400, 401, 403, 404
`POST`	`/employer/invitations/{id}/outcome`	Итог собеседования по принятому приглашению: наняли или не подошёл Сигнал для «Маринада»: голова «Подойдёт» ранжировщика учится на том, кого в итоге нанимают.	по входу	200, 400, 401, 403, 404
`GET`	`/candidate/invitations`	Входящие приглашения (видны только свои)	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/invitations/{id}/view`	Отметить как просмотренное	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/invitations/{id}/accept`	Принять — контакты раскрываются работодателю	по входу	200, 400, 401, 403, 404

`POST`	`/candidate/invitations/{id}/decline`	Отклонить с причиной в один клик (обратная связь работодателю)	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/invitations/{id}/salary-check`	«Ценник не врёт»: совпала ли зарплата на собеседовании с вилкой приглашения Если нет — у компании падает репутация, урезаются талоны и снимается бейдж «Честный ценник».	по входу	200, 400, 401, 403, 404

Маринад

Метод	Путь	Назначение	Доступ	Коды
`GET`	`/public/learning`	Как шаурматч_ учится: активные модели, накопленные сигналы, детектор «чёрствого лаваша», журнал версий Публичный паспорт моделей — прозрачность для кандидатов и работодателей. Кэш 30 секунд.	публично	200, 400, 404
`POST`	`/public/learning/recipe`	Попробуйте сами: разбор текста потребности словарём правил и собственной моделью шаурматч_	публично	200, 400, 404
`GET`	`/public/learning/titles`	Названия шаблонов и начинок	публично	200, 400, 404

Регулярные задачи

Метод	Путь	Назначение	Доступ	Коды
`POST`	`/employer/tasks`	Опубликовать задачу для категории по рецепту потребности	по входу	200, 400, 401, 403, 404
`GET`	`/employer/tasks`	Мои задачи и число решений	по входу	200, 400, 401, 403, 404

`GET`	`/employer/tasks/{id}/submissions`	Обезличенные решения («окошко ларька»): без имени, с результатом и рейтингом	по входу	200, 400, 401, 403, 404
`POST`	`/employer/tasks/{id}/submissions/{sid}/invite`	Пригласить автора понравившегося решения, не зная, кто это	по входу	200, 400, 401, 403, 404
`GET`	`/candidate/tasks`	Мои регулярные задачи: адресные от системы («Свежая начинка») и от работодателей для моей категории Если пора (не чаще раза в 7 дней), система сама создаёт адресную задачу по «остывшему» или слабому ингредиенту.	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/tasks/{id}/start`	Получить свой уникальный вариант задачи	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/tasks/{id}/submit`	Отправить решение (и подход, если спрашивают)	по входу	200, 400, 401, 403, 404

Тестирование

Метод	Путь	Назначение	Доступ	Коды
`POST`	`/candidate/tests`	Начать адаптивный тест на заявленный грейд Каждому кандидату — уникальные варианты (шаблон + мутации + обёртка) с заранее известной сложностью (LLTM). Правильный ответ получен исполнением кода. specialization — тест в другой специализации: так добавляется дополнительная категория (после подтверждения основной).	по входу	200, 400, 401, 403, 404

`GET`	`/candidate/tests`	Мои тестовые сессии	по входу	200, 400, 401, 403, 404
`GET`	`/candidate/tests/{id}`	Состояние сессии и текущее задание	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/tests/{id}/answer`	Ответить на текущее задание	по входу	200, 400, 401, 403, 404
`POST`	`/candidate/tests/{id}/abandon`	Прервать тест	по входу	200, 400, 401, 403, 404

Вакансии

Метод	Путь	Назначение	Доступ	Коды
`GET`	`/vacancies`	Опубликованные вакансии	публично	200, 400, 404
`GET`	`/vacancies/{id}`	Вакансия	публично	200, 400, 404
`POST`	`/vacancies/{id}/apply`	Откликнуться на вакансию (контакты раскрываются этой компании)	по входу	200, 400, 401, 403, 404
`GET`	`/candidate/applications`	Мои отклики и их статусы	по входу	200, 400, 401, 403, 404
`GET`	`/employer/vacancies`	Мои вакансии	по входу	200, 400, 401, 403, 404

`POST`	`/employer/vacancies`	Опубликовать вакансию (зарплата «от–до» обязательна)	по входу	200, 400, 401, 403, 404
`PUT`	`/employer/vacancies/{id}`	Редактировать или закрыть вакансию	по входу	200, 400, 401, 403, 404
`GET`	`/employer/vacancies/{id}/applications`	Отклики на вакансию (контакты откликнувшихся открыты)	по входу	200, 400, 401, 403, 404
`POST`	`/employer/applications/{id}/view`	Отметить отклик просмотренным	по входу	200, 400, 401, 403, 404
`POST`	`/employer/applications/{id}/accept`	Принять отклик	по входу	200, 400, 401, 403, 404
`POST`	`/employer/applications/{id}/decline`	Отклонить отклик	по входу	200, 400, 401, 403, 404

9. Библиотеки и компоненты

Всё ПО — с открытыми лицензиями (MIT, Apache-2.0, BSD, PostgreSQL, ISC).

Инфраструктура

Компонент	Версия	Лицензия	Назначение
PostgreSQL	16 (alpine)	PostgreSQL	Реляционная СУБД
nginx	1.27 (alpine)	BSD-2	Раздача SPA, прокси `/api`
Node.js	22	MIT	Backend, мок ФСП, исполнение JS-вариантов
Python	3.12	PSF	ШАУРМА-движок
Mailpit	latest	MIT	SMTP-заглушка и веб-интерфейс писем
Keycloak	26.0	Apache-2.0	ФСП ID (OIDC), по желанию

Backend (`backend/package.json`)

Пакет	Версия	Назначение
express	5.2.1	HTTP-сервер, REST
zod	4.6.5	Валидация запросов
@asteasolutions/zod-to-openapi	9.1.0	Генерация OpenAPI 3 из схем
swagger-ui-express	5.0.1	Swagger UI `/api/docs`
pg	8.23.1	Драйвер PostgreSQL
jsonwebtoken	9.0.3	JWT
bcryptjs	3.0.3	Хеширование паролей
cookie-parser	1.4.7	httpOnly-cookie
helmet	8.3.0	Заголовки безопасности
cors	2.8.6	CORS
express-rate-limit	8.7.0	Ограничение частоты запросов
multer	2.4.0	Загрузка PDF
nodemailer	10.0.15	Письма
pdfkit	0.20.2	PDF-профиль кандидата
typescript	6.0.3	Язык
tsx	4.23.15	Запуск TS в разработке

Frontend (`frontend/package.json`)

Пакет	Версия	Назначение
react, react-dom	19.3.0	UI
react-router-dom	7.18.4	Маршрутизация
vite	8.3.2	Сборка
@vitejs/plugin-react	6.1.2	JSX
typescript	7.0.2	Язык
Шрифты Montserrat, JetBrains Mono	Google Fonts (OFL)	Фирменный стиль ЛЦТ / ФСП

ШАУРМА-движок (`engine/requirements.txt`)

Пакет	Версия	Назначение
fastapi	0.136.3	HTTP API движка
uvicorn	0.49.0	ASGI-сервер
pydantic	2.11.10	Схемы
numpy	2.2.6	EAP-оценка в адаптивном тесте, валидация
pypdf	6.6.0	Извлечение текста из PDF-резюме
python-multipart	0.0.32	Загрузка файлов
httpx	0.28.1	Клиент GigaChat / YandexGPT
matplotlib	3.10.7	Графики отчёта валидации

Методы реализованы самостоятельно, без сторонних психометрических библиотек: Glicko-2 по статье Гликмана, 3PL/EAP, LLTM, индекс lz.

Мок ФСП (`fsp-mock/package.json`)

Пакет	Версия
express	5.2.1

10. Сборка, развёртывание и локальный запуск

Требования

- Docker 24+ и Docker Compose v2. Сервер: 2 vCPU, 4 ГБ RAM, 10 ГБ диска (без Keycloak хватит и 2 ГБ RAM).
- Порт 80 (или любой другой через `HTTP_PORT`) и 8025 для веб-интерфейса почты (по желанию).

Развёртывание на сервере

```
git clone <репозиторий> proof && cd proof
cp .env.example .env
#   PUBLIC_URL=http://<ip-или-домен>   — нужен для ссылок в письмах
#   JWT_SECRET=<длинная случайная строка>
#   HTTP_PORT=80
docker compose up -d --build
docker compose logs -f backend      # дождитесь строки «шаурматч_ API: …»
```

Проверка: - `http://<сервер>/` — приложение; - `http://<сервер>/api/docs` — Swagger; - `http://<сервер>/api/health` — `{"status":"ok"}`; - `http://<сервер>:8025` — входящие письма (Mailpit).

HTTPS. Поставьте перед контейнером `frontend` обратный прокси с сертификатом (Caddy, nginx + certbot) и укажите `PUBLIC_URL=https://…`. Cookie станут `Secure` автоматически.

Вход через ФСП ID (Keycloak), по желанию:

```
docker compose -f docker-compose.yml -f docker-compose.keycloak.yml up -d --build
```

Keycloak доступен на `:8180` (admin / `KEYCLOAK_ADMIN_PASSWORD`), демо-пользователь `fsp.demo` / `shaurma2026`. Если сервер не на localhost, задайте `KEYCLOAK_PUBLIC_URL=http://<сервер>:8180`.

Реальная почта. Задайте `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `SMTP_FROM` и `DEMO_MODE=false`: ссылка подтверждения перестанет показываться на экране.

LLM. `GIGACHAT_AUTH_KEY` или `YANDEX_API_KEY` + `YANDEX_FOLDER_ID` — по желанию.

Управление данными

```
docker compose exec backend node dist/seed.js --reset   # пересоздать демо-данные
```

```
docker compose down -v                                    # удалить всё, включая БД
```

Отключить автонаполнение: `SEED_ON_START=false`.

Локальный запуск без Docker (для разработки)

См. раздел «Локальная разработка» в README. Нужны Python 3.12+ и Node.js 22+ (Node используется движком для исполнения JS-вариантов).

Пересчёт валидации метода

```
cd engine && pip install -r requirements.txt
python -m validation.simulate --n 2000 --seed 42
# результат: docs/validation/report.md, metrics.json, *.png
```

Примечание для Windows

Если путь к проекту содержит кириллицу, `docker compose build` может упасть с ошибкой `x-docker-expose-session-sharedkey`. Запускайте `COMPOSE_BAKE=false docker compose up -d --build`.